```
load 'events.rb'

each_event do |event|
  each_setup do |setup|
    setup.call
  end
  puts "ALERT: #{event[:description]}" if event[:condition].call
end
```

The Shared Scope is contained in a lambda that is called immediately. The code in the lambda defines the RedFlag methods as *Kernel Methods (32)* that share two variables: setups and events. Nobody else can see these two variables, because they're local to the lambda. (Indeed, the only reason why we have a lambda here is that we want to make these variables invisible to anyone except the four Kernel Methods.) And yes, each call to Kernel.send is passing a block as an argument to another block.

Now those ugly global variables are gone, but the RedFlag code is not as pleasantly simple as it used to be. It's up to you to decide whether this change is an improvement or just an unwelcome obfuscation. While you decide that, there is one last change that is worth considering.

Adding a Clean Room

In the current version of RedFlag, events can change each other's shared top-level instance variables:

```
event "define a shared variable" do
  @x = 1
end
event "change the variable" do
  @x = @x + 1
end
```

You want events to share variables with setups, but you don't necessarily want events to share variables with each other. Once again, it's up to you to decide whether this is a feature or a potential bug. If you decide that events should be as independent from each other as possible (like tests in a test suite), then you might want to execute events in a *Clean Room (87)*:

blocks/redflag_5/redflag.rb

```
each_event do |event|
  env = Object.new
  each_setup do |setup|
    env.instance_eval &setup
  end
  puts "ALERT: #{event[:description]}" if env.instance_eval &(event[:condition])
end
```